# Minimal Example: Code Import with GitHub Hyperlinks

Template Example

January 29, 2026

## 1 Example: Importing Lean Code with GitHub Links

Here's the `GroupAction` class definition imported from the source file. Each line number is clickable and links to GitHub:

```
20  /-
21  Title: Defs.lean
22  Project: Project 1 (Group Actions)
23  Author: Frankie Feng-Cheng WANG
24  Date: June 2026
25  Copyright (c) 2026 Frankie Feng-Cheng WANG. All rights reserved.
26  Repository: https://github.com/FrankieeW/
27  -/
28  import Mathlib.Tactic
29  import Mathlib.Data.SetLike.Basic
30  import Mathlib.Data.Fintype.Card
31  import Mathlib.Algebra.Group.Basic
32  import Mathlib.Algebra.Group.Subgroup.Basic
33  import Mathlib.GroupTheory.Perm.Basic
34
35  /-! ## Definitions: group actions -/
36  /-- Defines a group action of a monoid `G` on a type `X`.
37  The action is given by `act : G → X → X`,
38  satisfying the axioms `ga_mul` and `ga_one`. -/
39  class GroupAction (G : Type*) [Monoid G] (X : Type*) where
40    act : G → X → X
41    ga_mul : ∀ g₁ g₂ x, act (g₁ * g₂) x = act g₁ (act g₂ x)
42    ga_one : ∀ x, act 1 x = x
```

**Try it:** Click any line number in the code block above to jump to that line on GitHub!

## 2 How It Works

1. `\leancodefile` imports code from a local `.lean` file
2. `firstline/lastline` specify which lines to extract

3. `firstnumber` sets the displayed line numbering
4. Each line is wrapped in `\href` linking to GitHub
5. The GitHub URL is constructed as: `base_url#LN` where N is the line number

# 3 Another Example: Permutation Representation

```
66  /-
67  Title: Permutation.lean
68  Project: Project 1 (Group Actions)
69  Author: Frankie Feng-Cheng WANG
70  Date: June 2026
71  Copyright (c) 2026 Frankie Feng-Cheng WANG. All rights reserved.
72  Repository: https://github.com/FrankieeW/
73  -/
74  import GroupAction.Defs
75
76  /-!
77  ## Theorem: permutation representation
78
79  Let `X` be a `G`-set.
80
81  1. For each `g ∈ G`, define `σ_g : X → X` by `σ_g(x) = g • x`.
82     This map is a permutation of `X` (a bijection).
83  2. Define `ψ : G → Sym X` by `ψ(g) = σ_g`.
84     This map is a group homomorphism.
85
86  Moreover, for all `g ∈ G` and `x ∈ X`, we have `ψ(g)(x) = g • x`.
87  -/
88
89  variable {G : Type*} [Group G] {X : Type*} [GroupAction G X]
90
91  /-- Defines the action map `σ_g : X → X` by `x ↦ g • x`. -/
92  def sigma (g : G) : X → X :=
93    fun x => GroupAction.act g x
94  #check sigma
95
96  /-- Defines the permutation of `X` induced by `g`.
97      The inverse is given by the action of `g⁻¹`. -/
98  def sigmaPerm (g : G) : Equiv.Perm X :=
99    { toFun := sigma g
100     invFun := sigma g⁻¹
101     left_inv := by
102       intro x
103       -- Step 1: combine `g⁻¹` and `g` using the action axiom.
104       calc
105         GroupAction.act g⁻¹ (GroupAction.act g x) =
106             GroupAction.act (g⁻¹ * g) x := by
107           simpa using (GroupAction.ga_mul g⁻¹ g x).symm
108         -- Step 2: simplify `g⁻¹ * g` to `1`.
109         _ = GroupAction.act (1 : G) x := by
110           -- Alternative: use `congrArg` with `inv_mul_cancel g`.
111           -- congrArg (fun t => GroupAction.act t x) (inv_mul_cancel g)
```

```
            simp
          -- Step 3: the identity acts as the identity on `X`.
          _ = x := GroupAction.ga_one x
      right_inv := by
        intro x
        -- Step 1: combine `g` and `g⁻¹` using the action axiom.
        calc
          GroupAction.act g (GroupAction.act g⁻¹ x) =
              GroupAction.act (g * g⁻¹) x := by
            simpa using (GroupAction.ga_mul g g⁻¹ x).symm
          -- Step 2: simplify `g * g⁻¹` to `1`.
          _ = GroupAction.act (1 : G) x := by
            -- Alternative: use `congrArg` with `mul_inv_cancel g`.
            -- congrArg (fun t => GroupAction.act t x) (mul_inv_cancel g)
            simp
          -- Step 3: the identity acts as the identity on `X`.
          _ = x := GroupAction.ga_one x }
#check sigmaPerm

/-- Defines the permutation representation `phi : G → Equiv.Perm X`.
    This sends `g` to the permutation `σ_g`. -/
def phi (g : G) : Equiv.Perm X :=
  sigmaPerm g
```